



INSI

RESTRUCTURATION DU BACKEND D'UNE PLATEFORME E-COMMERCE

Transition d'un Monolithe vers une Architecture Microservices

Mémoire de projet présenté en vue de l'obtention du diplôme de Master 2

Présenté par :

Tina Hubert RATOLOJANAHARY

Sous la direction de :

Membres du Jury de l'INSI

Sommaire de la Présentation

Introduction et Contexte

- ▶ Contexte et Problématique
- ▶ Objectifs de la Migration

Conception et Choix Techniques

- ▶ Modélisation DDD et Découpage
- ▶ Communication Inter-services (RabbitMQ)
- ▶ Persistance Polyglotte et Pattern Saga

Réalisation Technique

- ▶ Structure Monorepo et NestJS
- ▶ API Gateway et Sécurisation
- ▶ Tracing avec Jaeger et Observabilité

Résultats et Conclusion

- ▶ Bilan des Performances (Artillery)
- ▶ Analyse Statistique du Code (Lizard, LOC)
- ▶ Perspectives et Améliorations Futures

Contexte du Projet : Plateforme Techcare

- ▶ **Techcare** : Plateforme e-commerce officinale pour la vente en ligne de médicaments et produits de santé.
- ▶ **Le Point de Départ** : Une application monolithique traditionnelle construite en JavaScript/Node.js.
- ▶ **Le Défi** : Assurer la transition d'un monolithe vers une architecture orientée microservices moderne sans rupture de service et en garantissant la cohérence.

Architecture Initiale

- ▶ Base de code unique
- ▶ Base de données relationnelle unique
- ▶ Couplage fort entre fonctions

Problématique du Monolithe Legacy

Limites Techniques et Organisationnelles

- ▶ **"Big Ball of Mud"** : Code difficilement maintenable avec un risque élevé de régression.
- ▶ **Déploiements bloquants** : Une seule correction nécessite le redéploiement complet.
- ▶ **Absence de Scalabilité Ciblée** : Impossible de scaler le module de catalogue sans dupliquer la gestion des paiements.
- ▶ **SPOF (Single Point of Failure)** : Un crash mémoire dans un module paralyse toute la pharmacie.

Objectifs de la Migration

- ▶ **Isolation des pannes (Résilience).**
- ▶ **Scalabilité horizontale sélective.**
- ▶ **Indépendance des cycles de livraison.**
- ▶ **Charge cognitive réduite par service.**

Découpage Métier : Le Domain-Driven Design (DDD)

- ▶ Analyse approfondie des domaines métiers pour définir des **Bounded Contexts** (contextes délimités).
- ▶ Chaque microservice possède sa propre responsabilité métier claire.
- ▶ Utilisation de l'*Ubiquitous Language* (ex : `RESERVATION/RELEASE`, `stock_unavailable`) directement dans le code pour aligner technique et métier.

Les Contextes Retenus

Product Manager Catalogue de médicaments (NoSQL)

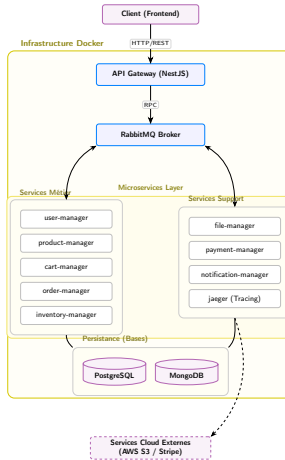
Inventory Manager Gestion fine des stocks (ACID)

Order Manager Gestion des paniers et commandes

User Manager Authentification et rôles (RBAC)

Support Services File Manager, Payment, Notification

Architecture des Conteneurs et Communications



Persistence Polyglotte : Le bon outil au bon endroit

Le choix des bases de données répond à des impératifs techniques divergents :

Critère	PostgreSQL (Inventory/Orders)	MongoDB (Product Catalog)
Modèle de données	Relationnel (Schéma fixe et strict)	Documentaire (Schéma dynamique)
Garanties	Forte cohérence (ACID)	Haute disponibilité (BASE)
Cas d'utilisation	Éviter le Double Spending lors des réservations de stock	Métadonnées hétérogènes (posologie, composition, photos)
Bénéfice	Transactions complexes fiables	Rapidité de consultation et flexibilité

Transactions Distribuées : Pattern Saga par Chorégraphie

Problématique

Chaque microservice ayant sa propre base de données, impossible d'effectuer un commit global ACID en cas d'erreur réseau ou applicative.

Solution : Saga Décentralisée

- ▶ Enchaînement d'événements asynchrones via RabbitMQ.
- ▶ Si une étape échoue (ex : paiement refusé), des **événements de compensation** sont émis pour annuler les étapes précédentes (ex : libération du stock réservé).

Déroulement d'une commande

1. validation du produit (NoSQL)
2. réservation de stock (PostgreSQL)
3. débit bancaire (Stripe)
4. confirmation (Email / WS)

Choix Technologiques et Structure du Code

- ▶ **TypeScript & NestJS** : Choix motivé par la rigueur du typage et le design pattern d'injection de dépendances (DI) facilitant l'architecture en couches (Clean/Hexagonal).
- ▶ **RabbitMQ** : Utilisé comme Message Broker asynchrone pour propager les événements métiers.
- ▶ **Shared Library** : Paquet centralisé hébergeant les DTOs, enums et filtres d'exceptions globaux (RpcExceptionFilter).

Structure du Projet

```
techcare-monorepo/  
  |- apps/  
    |- api-gateway/  
    |- order-manager/  
    +- ... (autres services)  
  +- libs/  
    +- shared/
```

Rôle de l'API Gateway

- ▶ Point d'entrée unique (BFF).
- ▶ Routage des requêtes vers les services.
- ▶ Validation centralisée des données (DTOs).
- ▶ Détection et interception des exceptions internes.

Sécurité Applicative (Principes Zero Trust)

- ▶ Authentification par jeton **JWT** au niveau de la Gateway.
- ▶ propagation de l'identité de l'utilisateur à travers les microservices via les headers de message.
- ▶ Contrôle d'accès par rôles (**RBAC**) sur les endpoints sensibles (Admin vs Client).

Scénario de Démonstration Live

Étape 1 : Cycle Nominal (Succès)

- ▶ **Achat Client** : Commande d'un produit.
- ▶ **Coulisses** : Validation → Réservation stock → Débit Stripe → Email.
- ▶ **UI** : Notification temps réel par WebSockets.

Étape 2 : Administration & RBAC

- ▶ Requête directe API par le Client (ex : Postman/cURL) → Rejetée (403 Forbidden).
- ▶ Connexion Admin sur l'interface → Ajout produit dans le catalogue

Étape 3 : Transaction de Compensation

- ▶ Commande client → Débit Stripe échoué.
- ▶ Déclenchement de la compensation → Libération automatique du stock réservé.

Étape 4 : Injection de Panne & Tracing

- ▶ Arrêt forcé d'une base de données ou d'un service.
- ▶ Analyse visuelle directe du flux d'erreur dans le tableau de bord **Jaeger**.

Analyse des Performances sous Stress (Artillery)

Un test de charge de **5 660 requêtes** sur 4 minutes (jusqu'à 100 requêtes/sec) a mis en évidence le comportement des systèmes :

- ▶ **Latence Nominale (Médiane)** : 29,1 ms pour les microservices contre 7 ms pour le monolithe.
- ▶ **"Impôt de la distribution"** : Surcharges liées au réseau (Network Hop), à la sérialisation JSON et au routage Gateway.
- ▶ **Résilience au stress** : Taux de succès de **86,5%** en microservices en pic de charge extrême (timeouts Gateway).

Latence de Queue (p95/p99)

- ▶ **p95** : 2,8 secondes.
- ▶ **p99** : 6,8 secondes.
- ▶ Preuve de la saturation des connexions socket en haute charge sans auto-scaling dynamique.

- ▶ **Volume de Code par dépôt :**
 - ▶ Monolithe global : **8 087 lignes** de code.
 - ▶ Service Produit isolé : **716 lignes** (-91%).
 - ▶ Charge cognitive drastiquement réduite pour les équipes.
- ▶ **Complexité Cyclomatique (Lizard) :**
 - ▶ Baisse du CCN moyen de **1,8** à **1,2**.
 - ▶ Moins de chemins d'exécution, réduction du risque de bugs logiques.

Indicateurs Lizard

Métrique	Mono	Micro
Fonctions	297	12
CCN Moy.	1.8	1.2
LOC / fonc.	15.6	3.8

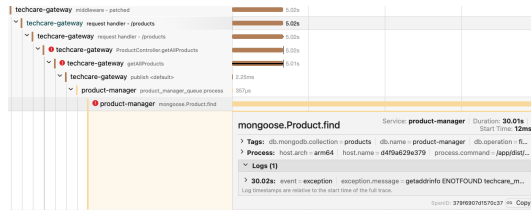
Observabilité et Isolation : Simulation de Panne

Scénario de test : MongoDB Down

Nous avons volontairement coupé MongoDB (catalogue produits) pour évaluer la résilience.

Constats

- ▶ **Dégradation Gracieuse** : Le catalogue devient indisponible, mais le reste (Paiements, Commandes) fonctionne.
- ▶ **Observabilité immédiate** : Grâce à **Jaeger** et **OpenTelemetry**, la cause (MongoDB Timeout) est identifiée en 2 secondes, sans fouiller les logs textuels de tous les conteneurs.



Trace Jaeger identifiant la défaillance

Bilan Quantitatif Global : Avant vs Après

Indicateur	Ancien Monolithe	Nouvelle Architecture
Temps de réponse (Médiane)	7 ms (Accès mémoire direct)	29,1 ms (Réseau + Routage Gateway)
Taux de succès sous stress	100 %	86,5 % (Saturation socket Gateway)
Temps de démarrage (Cache)	1,280 s (1 conteneur)	19,671 s (Orchestration multi-services)
Charge cognitive (LOC / dépôt)	8 087 lignes (Projet unique)	~ 700 lignes (Par microservice)
Complexité Cyclomatique (CCN)	1,8 (Moyenne)	1,2 (Moyenne - Single Responsibility)
Résilience au crash critique	Effondrement global du système	Dégradation gracieuse (Jaeger identifié)

Perspectives d'Améliorations Futures

DevOps & Infrastructure

- ▶ **Kubernetes (K8s)** : Auto-scaling horizontal automatique des pods lors des pics de charge, éliminant les échecs de connexion.
- ▶ **Service Mesh (Istio)** : Sécurité inter-services chiffrée (mTLS) et gestion intelligente du trafic réseau.

Optimisations Applicatives

- ▶ **Cache Redis** : Intégration d'une couche de cache au niveau de l'API Gateway pour faire chuter le temps de réponse de consultation (2 à 5 ms).
- ▶ **gRPC** : Remplacement du HTTP/JSON interne par gRPC/Protobuf binaire pour des communications RPC ultra-rapides.
- ▶ **Circuit Breakers** (ex : disjoncteurs) pour couper immédiatement les appels aux services saturés.

Conclusion Générale

- ▶ **Transition Réussie** : Une architecture opérationnelle, saine et industrialisée pour la plateforme Techcare.
- ▶ **Bénéfice Clé** : L'agilité opérationnelle et l'isolation des failles surpassent l'impôt de distribution lié aux performances réseau.
- ▶ **Compromis Accepté** : La migration vers les microservices n'est pas une solution miracle, elle nécessite une gestion rigoureuse de la complexité.

Merci pour votre attention

Avez-vous des questions ?